ENTREGA 12 · INNOVATION LAB

Integração Python Básica

Função customizada em Python para cálculo de progresso no Xano

Disciplina	Innovation Lab: Desenvolvimento Avançado No/Low Code
Módulo	2 – Integração Frontend-Backend
Pré-requisitos	Tarefa 11 (tabela `academic_tasks`)
Aluno	Wender Araújo Santos
E-mail institucional	`wender.araujo@aluno.impacta.edu.br`
Entrega (PR #10)	15 de outubro de 2026
Evidências	20 de outubro de 2026
Branch	`feat/python-progress`
Repositório	github.com/wenderaraujo-creator/EduTrack-IA-Wender-Est

Resumo. Uma função customizada em Python puro recebe as tarefas de uma disciplina em JSON e devolve o percentual de conclusão no mesmo formato — a base do cálculo de progresso do EduTrack AI. A mudança foi planejada com OpenSpec (`logica-calculo-progresso`), implementada em `scripts/calculate_progress.py`, testada localmente e arquivada, gerando a spec mestra `openspec/specs/progress-calculation/spec.md`.

SUMÁRIO

1. Objetivo e conceitos-chave
2. O fluxo OpenSpec (proposal → implementação → arquivo)
3. O script calculate_progress.py
4. Teste local e saída real
5. Evidências

6. Critérios de avaliação
7. Observações técnicas
8. Próximo passo

1 Objetivo e conceitos-chave

O objetivo é escrever a lógica de cálculo de progresso de uma disciplina em Python e empregá-la como *custom function* do Xano — o percentual de tarefas concluídas que alimentará o dashboard do app.

CONCEITO	O QUE SIGNIFICA NO PROJETO
Custom function	Função em Python hospedada no Xano, invocada por uma query. Recebe e devolve JSON.
Contrato JSON	Entrada `{"tasks": [...]}` → saída `{"percentage": ...}`.
Python restrito	Ambiente do Xano permite apenas *built-ins*; o script importa somente `json`.
Spec-Driven Development	A regra de cálculo foi especificada antes do código, em `progress-calculation/spec.md`.

2 O fluxo OpenSpec (proposal → implementação → arquivo)

A mudança `logica-calculo-progresso` seguiu os quatro artifacts do schema `spec-driven`:

ARTIFACT	ARQUIVO	PAPEL
`proposal`	`proposal.md`	Por quê e o que muda
`specs`	`specs/progress-calculation/spec.md`	Requisitos com cenários WHEN/THEN
`design`	`design.md`	Decisões (status canônico, divisão por zero, arredondamento)
`tasks`	`tasks.md`	Lista de execução

Validação e arquivamento:

```
openspec validate logica-calculo-progresso --strict
  →  Change 'logica-calculo-progresso' is valid

openspec archive logica-calculo-progresso
  →  openspec/changes/archive/2026-10-15-logica-calculo-progresso/
  →  openspec/specs/progress-calculation/spec.md  (criada)
```

3 O script calculate_progress.py

Três funções públicas e o ponto de entrada `handle(payload)`, usado pelo Xano:

```
def calculate_progress(completed_tasks, total_tasks):
    """Retorna {"percentage": x} de 0 a 100; 0 quando nao ha tarefas."""
    if total_tasks <= 0:
        return {"percentage": 0}
    return {"percentage": round(completed_tasks / total_tasks * 100, 2)}

def is_completed(task):
    """Uma tarefa conta como concluida quando o status e 'completed'."""
    status = str(task.get("status", "")).strip().lower()
    return status == COMPLETED_STATUS

def calculate_subject_progress(tasks):
    tasks = tasks or []
    total_tasks = len(tasks)
    completed_tasks = sum(1 for task in tasks if is_completed(task))
    return calculate_progress(completed_tasks, total_tasks)

def handle(payload):
    """Entrada/saida JSON: {"tasks": [...]} → {"percentage": ...}."""
    if isinstance(payload, str):
        payload = json.loads(payload)
    return calculate_subject_progress(payload.get("tasks", []))
```

DECISÃO	POR QUÊ
Só built-ins	O Xano executa funções em ambiente restrito; apenas `json` é importado.
Status canônico	Uma tarefa é concluída quando `status`, após `trim` e `lower`, é `completed`.
Divisão por zero	`total_tasks <= 0` devolve `{"percentage": 0}` em vez de levantar `ZeroDivisionError`.
Arredondamento	`round(..., 2)` — duas casas bastam para exibição e evitam ruído de ponto flutuante.

4 Teste local e saída real

O script foi executado localmente com quatro casos de exemplo (metade, total, vazio e status com espaços nas bordas):

```
$ python3 scripts/calculate_progress.py
{"percentage": 50.0}
{"percentage": 100.0}
{"percentage": 0}
{"percentage": 66.67}
```

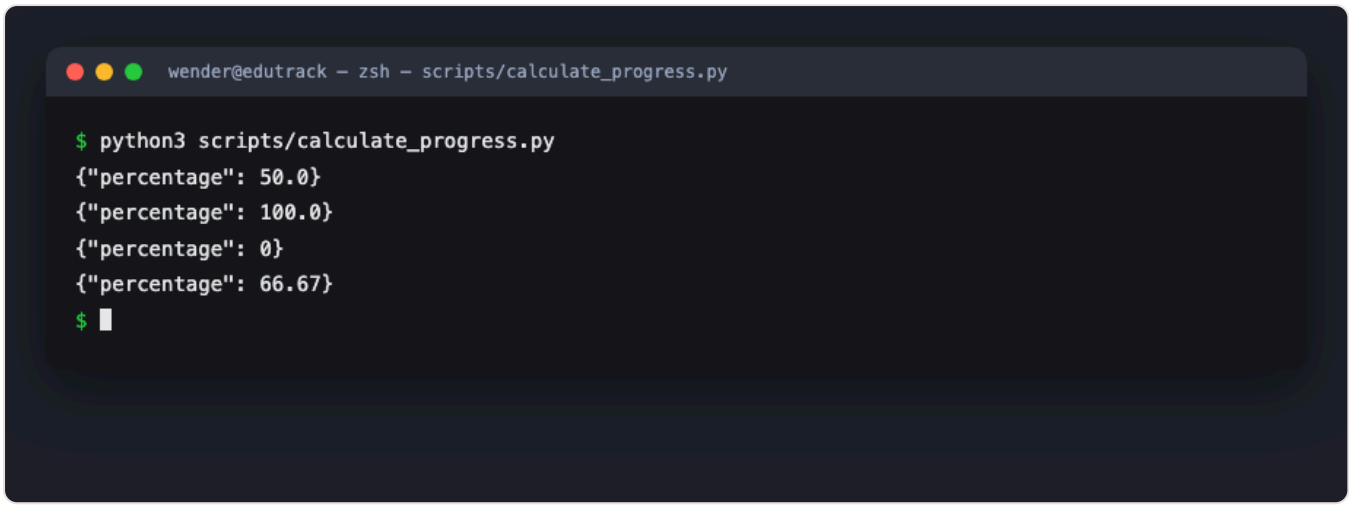


Figura 1 — Saída real do script no terminal. Os quatro casos de teste retornam `50.0`, `100.0`, `0` e `66.67`. A saída foi capturada no arquivo `saida-terminal.txt` e reproduzida nesta figura. `teste-saida-terminal.png`

CASO	ENTRADA	SAÍDA ESPERADA	SAÍDA REAL
Metade concluída	`1 completed, 1 pending`	`50.0`	✅ 50.0
Tudo concluído	`2 completed`	`100.0`	✅ 100.0
Sem tarefas	`[]`	`0`	✅ 0
Status com espaços	`" completed ", pending, completed`	`66.67`	✅ 66.67

5 Evidências

EVIDÊNCIA	ONDE
Script funcional	`scripts/calculate_progress.py` (PR #10 merged)
Change planejada e arquivada	`openspec/changes/archive/2026-10-15-logica-calculo-progresso/`
Spec mestra sincronizada	`openspec/specs/progress-calculation/spec.md`
Saída do terminal	`docs/evidencias/tarefa12/saida-terminal.txt`
Print do teste	`docs/evidencias/tarefa12/teste-saida-terminal.png`
Material de entrega	`docs/evidencias/tarefa12/Tarefa12-entrega.html` / `.pdf`

Verificação de integridade. `python3 -m py_compile scripts/calculate_progress.py` compila sem erros e a execução reproduz as quatro saídas esperadas — o mesmo resultado registrado no `docs/tarefas/tarefa12.md`.

6 Critérios de avaliação

CRITÉRIO	STATUS
Script Python funcional e sem erros de sintaxe	✅ `calculate_progress.py` compilado e executado
Uso correto do fluxo OpenSpec	✅ `logica-calculo-progresso` (4/4 artifacts, valid, arquivada)
Tratamento de erro (divisão por zero)	✅ retorna `{"percentage": 0}` quando não há tarefas
Commit na branch `feat/python-progress`	✅ PR #10 merged na `main`
Screenshot do teste do script	✅ `teste-saida-terminal.png`
Material de entrega versionado	✅ HTML + PDF em `docs/evidencias/tarefa12/`

7 Observações técnicas

TEMA	DECISÃO
Escopo	Somente o cálculo pedido. Sem endpoint REST (é a Tarefa 13), sem ligação às tabelas do Xano, sem tela.
Sem push pela IA	REGRA Nº 2 do `AGENTS.md`: a IA gera e testa os arquivos; publicar no Xano é passo manual do desenvolvedor.
Ambiente restrito	Custom functions do Xano rodam com *built-ins* apenas — por isso o script não usa bibliotecas externas.
Contrato de entrada	`handle` aceita payload já desserializado ou string JSON, cobrindo os dois formatos que o Xano pode enviar.
Branch de trabalho	`feat/python-progress`, PR #10 — merged na `main` em 15/10/2026.

8 Próximo passo

Tarefa 13: expor o cálculo de progresso e os endpoints REST das tabelas `subjects` e `user`, integrando o frontend ao backend do Xano.

Arquivos produzidos nesta tarefa. `scripts/calculate_progress.py` · `openspec/changes/archive/2026-10-15-logica-calculo-progresso/` (proposal, specs/progress-calculation, design, tasks) · `openspec/specs/progress-calculation/spec.md` · `docs/evidencias/tarefa12/` (este material)